

Mạng Nơ-ron Nhân Tạo & Tối Ưu Hóa Loss Function

Khám phá toàn diện kiến trúc Neural Network, phương pháp tính trọng số (Weights), thuật toán Lan truyền ngược (Backpropagation) và ví dụ tính toán chi tiết.

Phần 1: Giới Thiệu Neural Network

Tổng quan bản chất, định nghĩa và nguồn gốc cảm hứng sinh học của Mạng Nơ-ron Nhân Tạo (ANN).



Khái Niệm Mạng Nơ-ron Nhân Tạo



Định Nghĩa Mô Hình (ANN)

Mạng nơ-ron nhân tạo (Artificial Neural Network - ANN) là một hệ thống tính toán học máy lấy cảm hứng từ cấu trúc mạng lưới thần kinh sinh học trong bộ não con người. Nó có khả năng tự học biểu diễn dữ liệu từ kinh nghiệm.



Mục Tiêu Chính

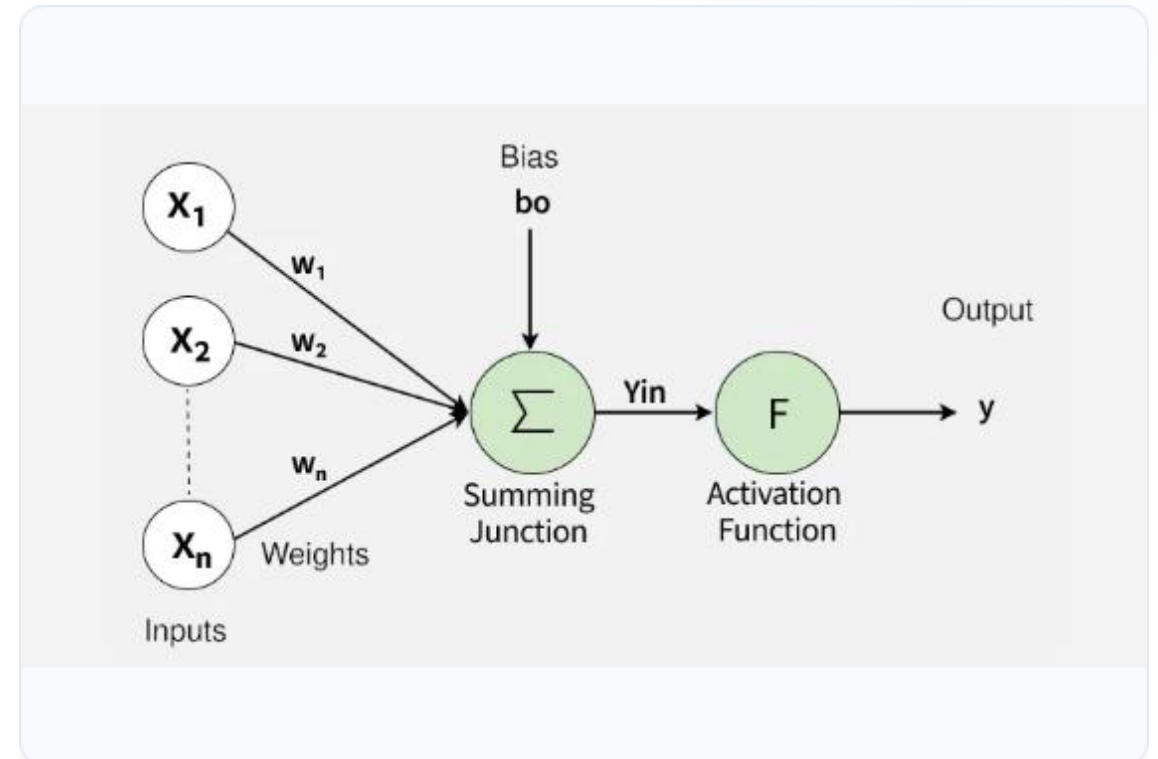
Xây dựng một hàm số xấp xỉ phức tạp $y = f(\mathbf{x}; \mathbf{W})$ nhằm xấp xỉ mối quan hệ giữa đầu vào $\mathbf{x}$ và đầu ra $\mathbf{y}$ bằng cách tự động điều chỉnh tập tham số trọng số $\mathbf{W}$ tối ưu.

Mô Hình Nơ-ron Sinh Học & Nhân Tạo

Sự Tương Đồng Về Cấu Trúc

Mỗi nút nơ-ron nhân tạo mô phỏng lại cách thức xử lý tín hiệu của một tế bào thần kinh sinh học:

- ✓ **Dendrites (Sợi nhánh):** Tương ứng với vector tín hiệu đầu vào $x_1, x_2, \dots, x_n$.
- ✓ **Synapses (Xơ-náp):** Tương ứng với các trọng số liên kết $w_1, w_2, \dots, w_n$.
- ✓ **Soma (Thân nơ-ron):** Thực hiện phép tổng đại số và cộng bias $z = \sum w_i x_i + b$.
- ✓ **Axon (Sợi trục):** Áp dụng hàm kích hoạt $a = \sigma(z)$ phát ra tín hiệu đầu ra.



CHƯƠNG 2

Phần 2: Thành Phần Cấu Trúc Chính

Tìm hiểu chi tiết các thành phần cốt lõi cấu thành nên một mạng nơ-ron hoàn chỉnh.

Các Tầng Trong Kiến Trúc Mạng



1. Input Layer

Tầng nhận dữ liệu thô đầu vào (các đặc trưng / features). Số lượng nơ-ron ở tầng này bằng đúng số chiều dữ liệu x.



2. Hidden Layer

Các tầng ẩn giữa input và output. Thực hiện biến đổi phi tuyến để trích xuất các đặc trưng phân cấp từ đơn giản đến phức tạp.



3. Output Layer

Tầng đầu ra đưa ra kết quả dự đoán cuối cùng (giá trị liên tục cho bài toán Regression hoặc xác suất cho Classification).

Trọng Số Weights Và Độ Lệch Bias

Trọng Số (Weights - W)

Đại diện cho cường độ kết nối giữa các nơ-ron. Trọng số đóng vai trò quyết định mức độ ảnh hưởng của một thuộc tính tín hiệu đầu vào đến kết quả của nơ-ron kế tiếp.

$$W = \begin{matrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{matrix}$$

Độ Lệch (Bias - b)

Là một tham số cộng thêm giúp dịch chuyển đường kích hoạt sang trái hoặc phải. Bias cho phép nơ-ron phát tín hiệu ngay cả khi tất cả các đầu vào bằng 0.

$$z = \sum_{i=1}^n w_i x_i + b$$

Vai Trò Của Hàm Kích Hoạt



Phi Tuyến Hóa

Nếu không có hàm kích hoạt, mạng nơ-ron dù nhiều tầng cũng chỉ là một phép biến đổi tuyến tính kết hợp $Y = Wn...W1 X + B$.



Xấp Xỉ Hàm Phức Tạp

Nhờ tính phi tuyến, mạng nơ-ron có khả năng xấp xỉ bất kỳ hàm số liên tục phức tạp nào (Universal Approximation Theorem).



Nén Khoảng Giá Trị

Giúp chuẩn hóa và giới hạn miền giá trị đầu ra của nơ-ron (ví dụ về khoảng $[0, 1]$ hoặc $[-1, 1]$) để tính toán ổn định.

So Sánh Các Hàm Kích Hoạt

Tên Hàm	Công Thức Toán Học	Miền Giá Trị	Ưu Điểm & Nhược Điểm Main
Sigmoid	$\sigma(z) = 1 / (1 + e^{-z})$	(0, 1)	Phù hợp đầu ra xác suất; dễ bị Triệt tiêu đạo hàm (Vanishing Gradient).
Tanh	$\tanh(z) = (e^z - e^{-z}) / (e^z + e^{-z})$	(-1, 1)	Đối xứng qua 0 giúp hội tụ nhanh hơn Sigmoid; vẫn bị Vanishing Gradient.
ReLU	$f(z) = \max(0, z)$	$[0, +\infty)$	Tính toán cực nhanh, giảm trệch gradient; có thể gặp hiện tượng Dying ReLU.
Leaky ReLU	$f(z) = \max(\alpha z, z)$ với $\alpha=0.01$	$(-\infty, +\infty)$	Khắc phục Dying ReLU bằng cách giữ gradient nhỏ cho giá trị âm.

Hàm Mất Mát Loss Function

Mean Squared Error (MSE)

Dùng cho bài toán Hồi quy (Regression). Đo lường trung bình bình phương khoảng cách giữa giá trị dự đoán $\hat{y}$ và thực tế y .

$$L = \frac{1}{2} \sum (y - \hat{y})^2$$

Binary Cross-Entropy (BCE)

Dùng cho bài toán Phân loại nhị phân (Binary Classification). Đo lường độ chênh lệch giữa hai phân phối xác suất.

$$L = -y \log(\hat{y}) - (1 - y) \log(1 - \hat{y})$$

CHƯƠNG 3

Phần 3: Forward Pass & Lan Truyền

Tìm hiểu chi tiết luồng dữ liệu tiến và quy trình tính toán qua từng nơ-ron trong mạng.

Quy Trình Lan Truyền Tiến

- **Bước 1: Tiếp nhận Input:** Vector dữ liệu đầu vào $\mathbf{X}$ được nạp vào Input Layer.
- 📊 **Bước 2: Tính tổng có trọng số:** Tại tầng l , tính tổng tuyến tính $z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$.
- ⚡ **Bước 3: Kích hoạt phi tuyến:** Tính đầu ra của tầng $a^{(l)} = \sigma(z^{(l)})$ thông qua hàm kích hoạt.
- 🚩 **Bước 4: Tính kết quả & Loss:** Tầng output đưa ra $\hat{y}$, sau đó so sánh với nhãn thực tế y để tính độ lỗi Loss $L(y, \hat{y})$.

Xu Hướng Giảm Loss Qua Các Epoch



Mô tả mức độ tổn thất (Loss) giảm dần và hội tụ khi tiến hành huấn luyện nhiều vòng (Epochs).

CHƯƠNG 4

Phần 4: Tối Ưu Hóa Loss Function

Tìm hiểu thuật toán Gradient Descent, Backpropagation và phương pháp cập nhật trọng số.



Tầm Quan Trọng Của Learning Rate

η

Learning Rate (α)

Siêu tham số quyết định bước nhảy (Step Size)

Learning rate quy định mức độ thay đổi của trọng số sau mỗi bước cập nhật gradient:

-  **Quá lớn (> 0.5):** Bước nhảy quá xa khiến Loss nhảy vọt, mô hình bị phân kỳ (Divergence).
-  **Quá nhỏ (< 0.0001):** Mạng hội tụ cực kỳ chậm hoặc dễ bị mắc kẹt tại cực trị địa phương (Local Minima).
-  **Mức tối ưu ($0.001 - 0.01$):** Giúp mô hình tiến dần và ổn định về điểm tối ưu toàn cục.

Các Thuật Toán Tối Ưu Hóa



SGD

Stochastic Gradient Descent: Cập nhật trọng số theo từng mẫu đơn lẻ. Nhanh nhưng đường đi đến cực trị bị nhiễu (noisy).



SGD + Momentum

Tích lũy gia tốc từ các bước gradient trước đó, giúp vượt qua các điểm yên ngựa (Saddle points) và vùng bằng phẳng.



Adam Optimizer

Kết hợp giữa AdaGrad và RMSProp. Tự động điều chỉnh learning rate cho từng tham số, được sử dụng phổ biến nhất hiện nay.

"Huấn luyện Mạng nơ-ron chính là hành trình tìm đường leo xuống thung lũng sâu nhất trên bề mặt không gian tổn thất nhiều chiều."

— Nguyên lý cốt lõi của Optimization trong Deep Learning

CHƯƠNG 5

Phần 5: Ví Dụ Tính Toán Chi Tiết

Thực hành tính toán số liệu cụ thể cho từng bước: Forward Pass, Loss, và Backpropagation Weight Update.

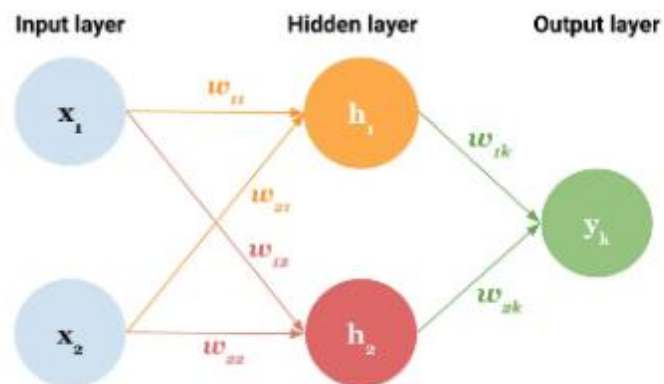
Kiến Trúc Mạng Ví Dụ Mẫu

Cấu Hình Mạng Đơn Giản

Xét mô hình mạng nơ-ron gồm 1 Input, 1 Hidden Neuron và 1 Output Neuron:

- **Đầu vào (Input x):** $x = 1.5$
- 🎯 **Giá trị thực tế (Target y):** $y = 0.8$
- 📐 **Hàm kích hoạt:** Sigmoid $\sigma(z) = 1 / (1 + e^{-z})$
- 📊 **Hàm Mất Mát:** $MSE L = 0.5 * (y - \hat{y})^2$
- 🎓 **Tốc độ học (Learning Rate):** $\eta = 0.1$

Illustrative example of a neural network



x_1, x_2 : input data features
 w_i : weights of the network
 h_1, h_2 : nodes in the hidden layer
 y_k : output variable

© AIML.com Research

This example consists of an input layer with two input nodes, one hidden layer with two neurons and an output layer with one neuron (eg: binary classification problem)

Thông Số Khởi Tạo Ban Đầu

Tầng / Tham Số	Ký Hiệu	Giá Trị Ban Đầu	Mô Tả Chức Năng
Weight Tầng Ẩn	w_1	0.40	Trọng số kết nối Input x vào Hidden Neuron h
Bias Tầng Ẩn	b_1	0.10	Độ lệch của Hidden Neuron h
Weight Tầng Output	w_2	0.60	Trọng số kết nối Hidden Neuron h vào Output y
Bias Tầng Output	b_2	0.20	Độ lệch của Output Neuron

Tính Toán Bước 1 Forward Pass



1. Tính tại Hidden Layer (h):

$$z_h = w_1 * x + b_1 = 0.40 * 1.5 + 0.10 = 0.60 + 0.10 = 0.70$$

$$a_h = \sigma(z_h) = 1 / (1 + e^{-0.70}) \approx 0.6682$$



2. Tính tại Output Layer (y):

$$z_{out} = w_2 * a_h + b_2 = 0.60 * 0.6682 + 0.20 = 0.4009 + 0.20 = 0.6009$$

$$\hat{y} = a_{out} = \sigma(z_{out}) = 1 / (1 + e^{-0.6009}) \approx 0.6458$$

Tính Toán Bước 2 Đánh Giá Loss



Kết quả dự đoán ($\hat{y}$): $\hat{y} \approx 0.6458$



Giá trị mục tiêu (y): $y = 0.8000$



Tính giá trị Loss Function (MSE):

$$L = 0.5 * (y - \hat{y})^2 = 0.5 * (0.8000 - 0.6458)^2$$

$$L = 0.5 * (0.1542)^2 = 0.5 * 0.023778 \approx 0.01189$$



Nhận xét: Mức độ sai lệch hiện tại là 0.01189. Chúng ta cần lan truyền ngược để cập nhật các trọng số w_1, w_2 sao cho Loss giảm tiếp.

Bước 3 Backpropagation Chain Rule



Áp dụng Đạo hàm chuỗi (Chain Rule) tính $\partial L / \partial w_2$:

$$\partial L / \partial w_2 = (\partial L / \partial \hat{y}) * (\partial \hat{y} / \partial z_{\text{out}}) * (\partial z_{\text{out}} / \partial w_2)$$



Tính từng thành phần đạo hàm riêng:

1. $\partial L / \partial \hat{y} = -(y - \hat{y}) = -(0.8000 - 0.6458) = -0.1542$

2. $\partial \hat{y} / \partial z_{\text{out}} = \hat{y} * (1 - \hat{y}) = 0.6458 * (1 - 0.6458) \approx 0.2287$

3. $\partial z_{\text{out}} / \partial w_2 = a_h = 0.6682$



Tổng hợp Đạo hàm Gradient $\partial L / \partial w_2$:

$$\partial L / \partial w_2 = (-0.1542) * 0.2287 * 0.6682 \approx -0.02356$$

Bước 4 Cập Nhật Trọng Số Mới

Tham Số	Giá Trị Cũ	Gradient ($\partial L / \partial W$)	Công Thức Cập Nhật ($\eta = 0.1$)	Giá Trị Mới
w_2	0.6000	-0.02356	$w_{2_new} = 0.60 - 0.1 * (-0.02356)$	0.60236
b_2	0.2000	-0.03527	$b_{2_new} = 0.20 - 0.1 * (-0.03527)$	0.20353
w_1	0.4000	-0.00318	$w_{1_new} = 0.40 - 0.1 * (-0.00318)$	0.40032
b_1	0.1000	-0.00212	$b_{1_new} = 0.10 - 0.1 * (-0.00212)$	0.10021

HOÀN THÀNH BÀI TRÌNH BÀY

Cảm Ơn & Thảo Luận (Q&A)

Chúng ta đã hoàn thành bài tổng quan về Neural Network, cách tính Weights, Loss Function & Gradient Descent.

Xin mời quý thầy cô và các bạn đặt câu hỏi!

Image Sources



https://media.geeksforgeeks.org/wp-content/uploads/20250822111100486293/activation_function.webp

Source: www.geeksforgeeks.org



<https://aiml.com/wp-content/uploads/2023/08/Illustration-of-a-neural-net.png>

Source: aiml.com